



INFORME 1

Sanos y Salvos

Docente: KRYSTIAN REYMON DELGADO GUZMAN

Integrantes: Yamil Apablaza
Álvaro Del Canto
Santiago Vargas

ASIGNATURA: FULL STACK III

FECHA: 8 DE ABRIL DEL 2026

Introducción	4
Selección de Patrones de Arquitectura y Justificación	4
Patrones de Arquitectura Utilizados	4
1. Patrón de Microservicios	5
Microservicios principales	5
Justificación	5
2. Patrón API Gateway	6
Justificación	6
3. Repository Pattern	6
Justificación	6
4. Factory Method	7
Justificación	7
5. Circuit Breaker	7
Justificación	7
Tecnologías Utilizadas	7
Backend: Django y Django REST Framework	8
Justificación	8
Aporte a la eficiencia	8
Frontend: React con enfoque PWA	8
Justificación	8
Aporte a la eficiencia	8
Base de Datos: PostgreSQL con PostGIS	8
Justificación	9
Aporte a la eficiencia	9
Geolocalización: Mapbox y Leaflet	9
Justificación	9
Aporte a la eficiencia	9
Integración de APIs externas: Dog API	9
Justificación	9
Aporte a la eficiencia	10
Inteligencia Artificial: Google Cloud Vision API	10
Justificación	10
Aporte a la eficiencia	10
Justificación	10
Aporte a la eficiencia	10
Evaluación del Diseño frente a los Requerimientos del Cliente	10
Cumplimiento de Requerimientos Funcionales	11
Registro de mascotas	11
Visualización geográfica	11
Detección de coincidencias	11
Comunicación entre usuarios	11
Notificaciones automáticas	12

Arquitectura de microservicios	12
Uso de patrones de diseño	12
Integración de APIs externas	12
Escalabilidad y rendimiento	13
Consideraciones de Seguridad, Privacidad y Sostenibilidad	13
Seguridad	13
Privacidad	13
Sostenibilidad	14

Introducción

El informe propone una solución para la problemática de mascotas perdidas mediante una arquitectura de microservicios, que permite centralizar la información, mejorar la gestión de reportes y facilitar la localización de animales. El sistema integra geolocalización, análisis de imágenes y comunicación entre usuarios para aumentar la probabilidad de recuperación.

Selección de Patrones de Arquitectura y Justificación

Además, se definen patrones de arquitectura y tecnologías que aseguran una solución escalable, eficiente y alineada con los requerimientos del cliente, considerando también aspectos de seguridad, privacidad y sostenibilidad.

Patrones de Arquitectura Utilizados

La arquitectura incorpora los siguientes patrones:

- Microservicios
- API Gateway
- Repository Pattern
- Factory Method
- Circuit Breaker

Estos patrones se integran dentro de una arquitectura distribuida donde cada servicio cumple una función específica, logrando **alta cohesión y bajo acoplamiento entre los componentes del sistema**.

1. Patrón de Microservicios

El patrón de microservicios divide el sistema en servicios independientes responsables de funcionalidades específicas.

Microservicios principales

- **Auth Service:** autenticación de usuarios
- **User Service:** gestión de usuarios
- **Pet Service:** gestión de mascotas
- **Geo Service:** geolocalización de reportes
- **Match Service:** detección de coincidencias
- **Chat Service:** comunicación entre usuarios
- **Notification Service:** envío de notificaciones
- **Media Service:** gestión de imágenes

Justificación

- Permite **separar responsabilidades** dentro del sistema.
- Facilita la **escalabilidad de módulos críticos** como coincidencias o geolocalización.
- Mejora la **mantenibilidad**, permitiendo modificar servicios sin afectar todo el sistema.
- Permite integrar servicios externos como **APIs de inteligencia artificial**.

Este enfoque permite que la plataforma evolucione de forma flexible incorporando nuevas funcionalidades.

2. Patrón API Gateway

El **API Gateway** actúa como punto de entrada único para las solicitudes provenientes del frontend.

Justificación

- Centraliza las peticiones hacia los microservicios.
- Simplifica la comunicación entre cliente y backend.
- Permite implementar **seguridad mediante autenticación y tokens JWT**.
- Reduce la complejidad del cliente evitando llamadas directas a múltiples servicios.

Este patrón organiza el acceso a los recursos del sistema y mejora la gestión de las comunicaciones.

3. Repository Pattern

El **Repository Pattern** separa la lógica de negocio del acceso a datos.

Justificación

- Mantiene el código **organizado y mantenible**.
- Mejora la **testabilidad** mediante repositorios simulados.

Este patrón se utiliza para gestionar entidades como:

- Mascotas
- Reportes
- Ubicaciones geográficas

4. Factory Method

El **Factory Method** permite crear objetos de forma flexible sin depender directamente de clases específicas.

Justificación

- Permite crear distintos tipos de reportes (mascota perdida o encontrada).
- Facilita agregar nuevas categorías de mascotas en el futuro.

Este patrón mejora la **extensibilidad y adaptabilidad** del sistema.

5. Circuit Breaker

El **Circuit Breaker** permite manejar fallos en la comunicación con servicios externos.

Justificación

- Evita **fallos en cascada** cuando un servicio externo falla.
- Mejora la **resiliencia del sistema**.

Es especialmente importante debido al uso de **APIs externas de inteligencia artificial para análisis de imágenes**.

Tecnologías Utilizadas

Backend: Django y Django REST Framework

Se utiliza **Django** junto con **Django REST Framework (DRF)** para desarrollar los microservicios.

Justificación

- Facilita la creación de **APIs REST estructuradas**.
- Incluye mecanismos de autenticación y control de acceso.

Aporte a la eficiencia

- Mejora la organización del código.
- Permite construir servicios escalables.

Frontend: React con enfoque PWA

La interfaz se desarrolla con **React** bajo el enfoque de **Progressive Web App (PWA)**.

Justificación

- Arquitectura basada en **componentes reutilizables**.

- Permite instalar la aplicación en dispositivos móviles sin desarrollo nativo.

Aporte a la eficiencia

- Reduce costos de desarrollo.
- Permite interacción fluida con los microservicios.

Base de Datos: PostgreSQL con PostGIS

Se utiliza **PostgreSQL** con la extensión **PostGIS** para manejar datos geográficos.

Justificación

- Base de datos robusta y escalable.
- Permite almacenar y consultar **datos geoespaciales**.
- Facilita consultas de proximidad.

Aporte a la eficiencia

- Optimiza consultas de geolocalización.
- Permite detectar mascotas cercanas a una ubicación.
- Mejora el rendimiento del sistema.

Geolocalización: Mapbox y Leaflet

Se utilizan **Mapbox** para mapas y **Leaflet** para renderización.

Justificación

- Integración sencilla con aplicaciones web.
- Manejo eficiente de datos geográficos.

Aporte a la eficiencia

- Mejora la visualización de reportes.
- Permite mapas interactivos en tiempo real.

Integración de APIs externas: Dog API

Se utiliza **Dog API** para obtener información sobre razas.

Justificación

- Proporciona información estructurada y actualizada.

Aporte a la eficiencia

- Reduce complejidad del sistema.
- Mejora la calidad de los datos.

Inteligencia Artificial: Google Cloud Vision API

Se utiliza **Google Cloud Vision API** para analizar imágenes de mascotas.

Justificación

- Detecta características visuales automáticamente.
- Permite comparar imágenes entre reportes.

Aporte a la eficiencia

- Automatiza el proceso de coincidencias.
- Aumenta la probabilidad de identificar mascotas.

Comunicación en tiempo real: WebSockets

Los **WebSockets** permiten implementar el chat entre usuarios.

Justificación

- Comunicación bidireccional en tiempo real.
- Ideal para sistemas de mensajería.

Aporte a la eficiencia

- Mensajes instantáneos.
- Mejora la interacción entre usuarios.

Evaluación del Diseño frente a los Requerimientos del Cliente

Cumplimiento de Requerimientos Funcionales

Registro de mascotas

El **Pet Service** gestiona información de mascotas perdidas o encontradas incluyendo:

- características físicas
- imágenes

Esto permite centralizar información que actualmente se encuentra dispersa.

Visualización geográfica

El **Geo Service**, junto con Mapbox y Leaflet, permite:

- visualizar reportes en un mapa interactivo
- identificar mascotas cercanas

Detección de coincidencias

El **Match Service**, apoyado por Google Cloud Vision API, permite:

- analizar imágenes de mascotas
- comparar reportes

Esto automatiza un proceso que normalmente se realiza manualmente en redes sociales.

Comunicación entre usuarios

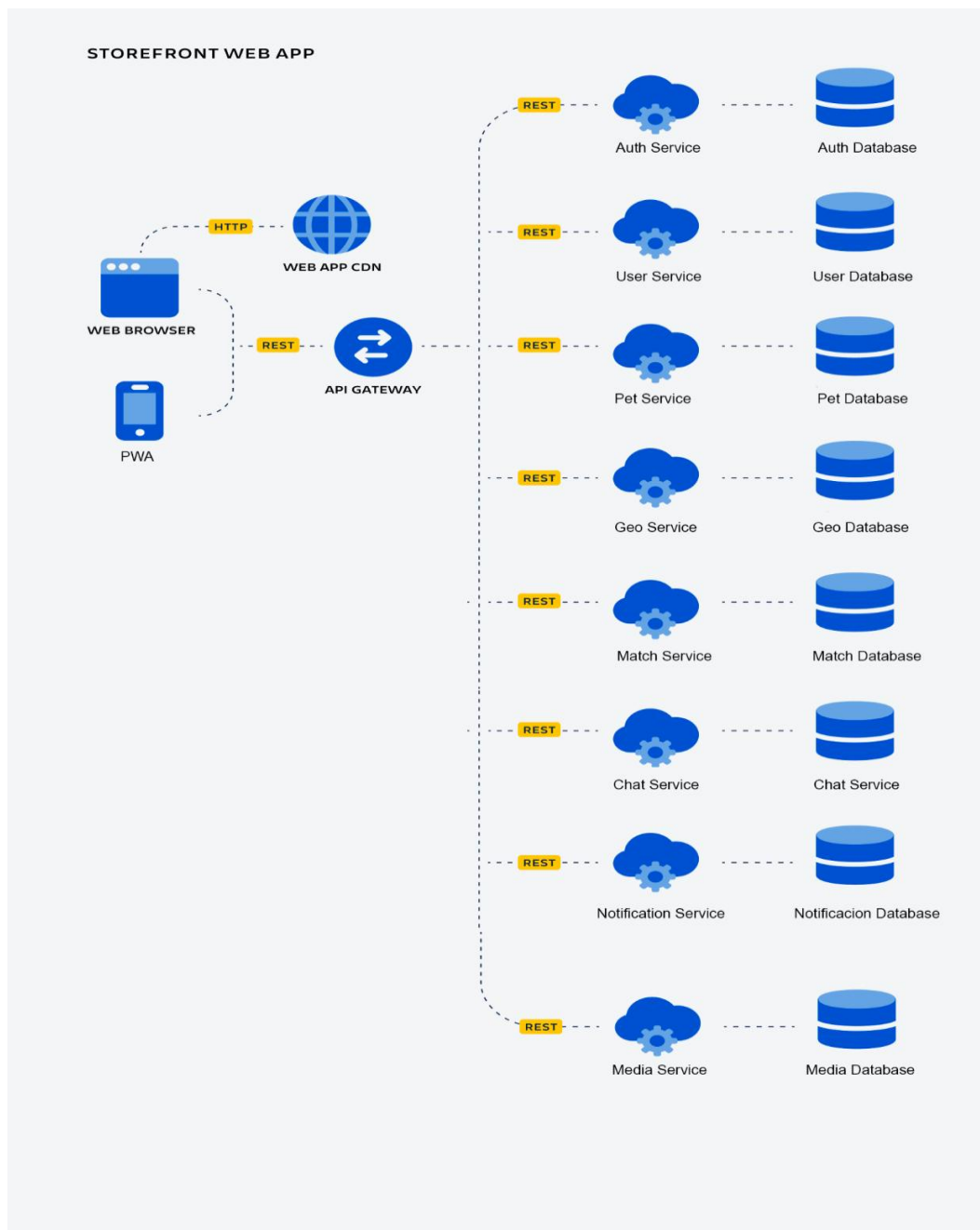
El **Chat Service** permite que usuarios se comuniquen cuando existe un posible match, facilitando la coordinación para la recuperación de mascotas.

Notificaciones automáticas

El **Notification Service** informa a los usuarios cuando el sistema detecta coincidencias, mejorando la rapidez del proceso.

Cumplimiento de Especificaciones Técnicas

Arquitectura de microservicios



Permite:

- escalar servicios de forma independiente
- mejorar resiliencia del sistema
- facilitar mantenimiento

Uso de patrones de diseño

La arquitectura incorpora:

- API Gateway
- Repository Pattern
- Factory Method
- Circuit Breaker

Estos patrones permiten estructurar el sistema siguiendo buenas prácticas de la industria.

Integración de APIs externas

El sistema utiliza:

- **Dog API** para información de razas
- **Google Vision API** para análisis de imágenes

Esto amplía las funcionalidades sin aumentar significativamente la complejidad.

Escalabilidad y rendimiento

La arquitectura permite escalar servicios con mayor demanda, especialmente el **Match Service**, que requiere mayor capacidad por el análisis de imágenes.

Consideraciones de Seguridad, Privacidad y Sostenibilidad

Seguridad

La plataforma incorpora medidas como:

- autenticación mediante **tokens JWT**
- comunicación segura mediante **HTTPS**

- validación de datos en Django
- separación de servicios mediante microservicios

Esto protege el acceso a la plataforma y reduce riesgos de seguridad.

Privacidad

El sistema gestiona de forma controlada datos sensibles como:

- información de contacto
- ubicación geográfica

Estos datos se utilizan únicamente para facilitar la recuperación de mascotas.

Sostenibilidad

La arquitectura utiliza tecnologías modernas que permiten optimizar recursos:

- microservicios escalables
- uso de herramientas consolidadas como Django, React y PostgreSQL

Esto facilita el mantenimiento del sistema y su evolución futura.

Continuación del Informe —

Evaluación Parcial 2

Implementación de Componentes Frontend y Backend

Durante la segunda etapa del proyecto se desarrollaron e integraron nuevos componentes frontend y backend con el objetivo de construir una solución distribuida basada en microservicios, aplicando patrones arquitectónicos, estrategias de branching y control de versiones mediante GitHub.

La solución fue evolucionando desde el análisis arquitectónico inicial hacia una implementación funcional que incorpora autenticación, geolocalización, administración y comunicación entre servicios.

Componentes Backend Implementados

La arquitectura backend fue construida bajo una estructura de microservicios independientes comunicados mediante APIs REST.

1. Auth Service

El Auth Service fue implementado como microservicio encargado de la autenticación y autorización de usuarios.

Funcionalidades implementadas

- Inicio de sesión mediante JWT.
- Generación de Access Token y Refresh Token.
- Validación de credenciales.
- Protección de endpoints privados.
- Integración con Django REST Framework y SimpleJWT.

Tecnologías utilizadas

- Django
- Django REST Framework
- SimpleJWT

Justificación

La separación del servicio de autenticación permite centralizar la seguridad del sistema y desacoplar el manejo de usuarios del resto de funcionalidades.

Beneficios

- Mayor seguridad.
- Escalabilidad independiente.
- Reutilización del sistema de autenticación por otros servicios.

2. User Service

El User Service fue implementado como microservicio encargado de la gestión de usuarios del sistema.

Funcionalidades implementadas

- Registro de usuarios.
- CRUD de usuarios.
- Panel administrativo mediante Django Admin.
- Protección mediante JWT.
- Serialización de datos usando DRF.

Tecnologías utilizadas

- Django
- Django REST Framework
- SQLite durante desarrollo

Justificación

Separar la gestión de usuarios facilita el mantenimiento y permite administrar la información de manera independiente al sistema de autenticación.

Beneficios

- Organización modular.
- Mejor mantenibilidad.
- Facilidad para futuras ampliaciones.

3. Geo Service

El Geo Service fue incorporado como microservicio especializado en geolocalización.

Funcionalidades implementadas

- Manejo de ubicaciones geográficas.
- Recepción de coordenadas.
- Integración futura con mapas interactivos.
- Base para la visualización de reportes geográficos.

Tecnologías utilizadas

- Django REST Framework
- APIs de geolocalización

Justificación

La geolocalización es uno de los componentes principales del sistema debido a la necesidad de ubicar mascotas perdidas o encontradas.

Beneficios

- Centralización de datos geográficos.

- Escalabilidad independiente.
- Optimización futura de consultas espaciales.

Backend For Frontend (BFF)

Se implementó un Backend For Frontend (BFF) como capa intermedia entre el frontend y los microservicios.

Funcionalidades del BFF

- Centralización de peticiones.
- Simplificación de consumo desde React.
- Comunicación con Auth Service y User Service.
- Manejo centralizado de respuestas.

Justificación

El patrón BFF permite adaptar la información específicamente para el frontend, reduciendo complejidad y desacoplando la interfaz de los microservicios internos.

Beneficios

- Menor acoplamiento.
- Mejor organización.
- Facilidad para escalar el frontend.
- Mayor control sobre seguridad y validaciones.

Implementación Frontend

El frontend fue desarrollado utilizando React bajo una arquitectura basada en componentes reutilizables.

Componentes implementados

- LoginSection
- Modal de autenticación
- Formularios de registro
- Manejo de sesiones mediante JWT
- Consumo de APIs REST mediante fetch

Integración con Backend

El frontend fue conectado a los microservicios mediante peticiones HTTP:

Auth Service

- Endpoint login/
- Validación de credenciales
- Obtención de JWT

User Service

- Endpoint admin/
- Registro de usuarios
- Gestión de datos de usuario

Manejo de Tokens JWT

Los tokens JWT son almacenados en LocalStorage para mantener la sesión del usuario autenticado.

Beneficios

- Persistencia de sesión.
- Protección de endpoints privados.
- Comunicación segura entre frontend y backend.

Estrategia de Branching

Para el desarrollo colaborativo del proyecto se utilizó una estrategia de branching basada en Git y GitHub.

Branch principal

- main

Objetivos de la estrategia

- Separar funcionalidades.
- Evitar conflictos entre integrantes.
- Facilitar integración continua.
- Permitir pruebas independientes.

Beneficios

- Mejor organización del trabajo.
- Mayor control de cambios.
- Trabajo paralelo entre integrantes.

Uso de GitHub y Versionamiento

Todos los componentes fueron versionados mediante GitHub.

Repositorios implementados

- Repositorio frontend React.
- Repositorio Auth Service.
- Repositorio User Service.
- Repositorio Geo Service.

Beneficios del versionamiento

- Historial de cambios.

- Trabajo colaborativo.
- Control de versiones.
- Respaldo del proyecto.

Seguridad Implementada

La solución incorpora mecanismos de seguridad modernos.

Medidas implementadas

- Autenticación JWT.
- Protección de endpoints.
- Variables sensibles mediante .env.
- Exclusión de credenciales mediante .Gitignore.
- Validación de datos desde backend.

Beneficios

- Protección de información sensible.
- Prevención de accesos no autorizados.
- Mejor gestión de configuraciones.

Evaluación Técnica de la Implementación

La implementación desarrollada cumple con los requerimientos técnicos solicitados en la evaluación.

Requerimientos cumplidos

- Arquitectura de microservicios.
- Implementación de Backend For Frontend.
- Uso de componentes frontend.
- Uso de JWT.
- Integración frontend-backend.
- Uso de GitHub.
- Estrategia de branching.
- Aplicación de patrones arquitectónicos.
- Separación de responsabilidades.

Conclusión

La segunda etapa del proyecto permitió transformar el diseño arquitectónico inicial en una solución funcional basada en microservicios y componentes frontend integrados.

La incorporación de Auth Service, User Service, Geo Service y el Backend For Frontend permitió mejorar la organización, escalabilidad y seguridad del sistema. Además, el uso de JWT, React, GitHub y estrategias de branching fortaleció el desarrollo colaborativo y la mantenibilidad de la plataforma.

La arquitectura implementada entrega una base sólida para continuar integrando nuevos servicios como Match Service, Notification Service y Chat Service en futuras etapas del proyecto.